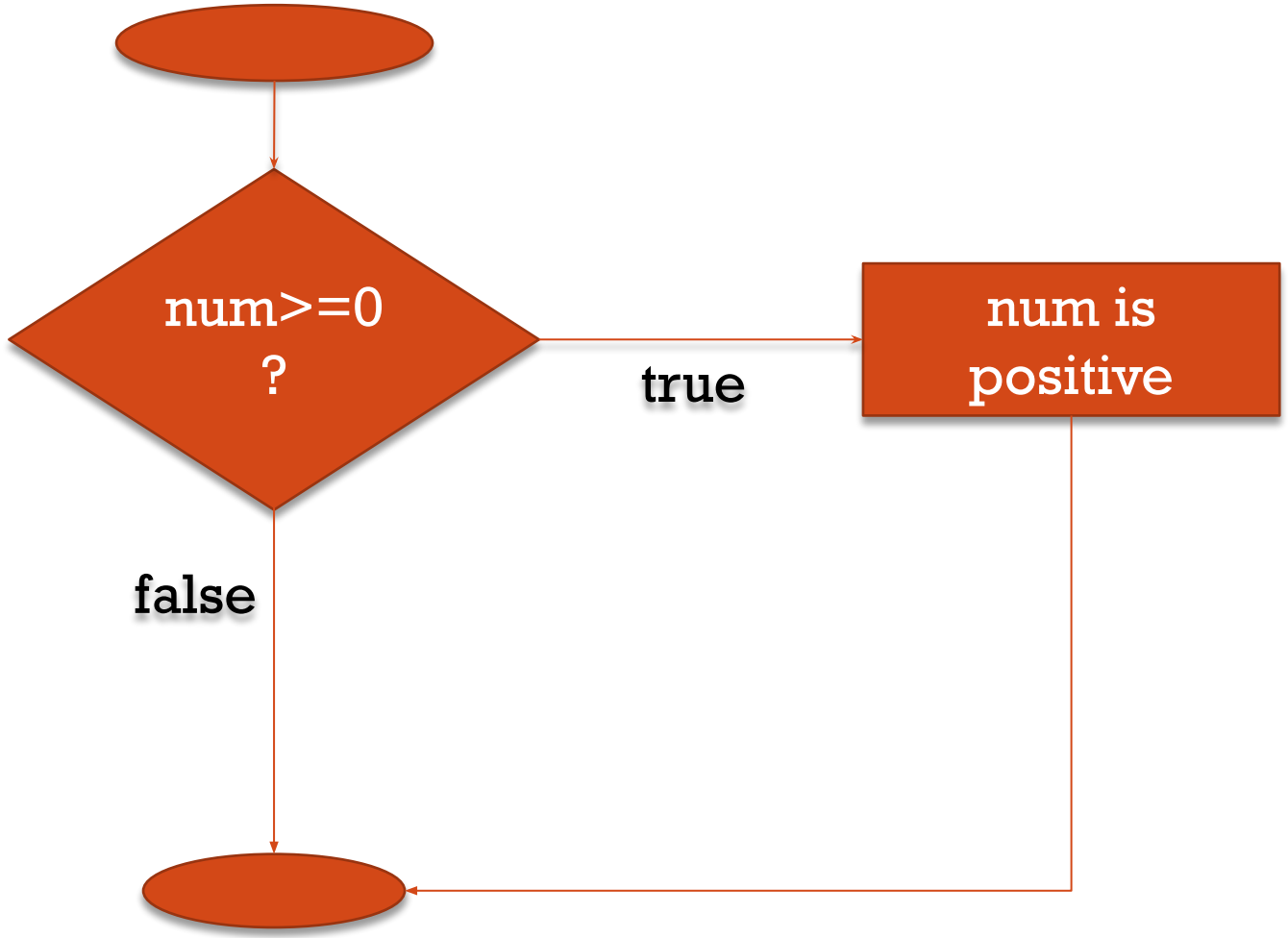
CONTROL STATEMENTS

Lecture: 5,6

Reference: Chapter 2.1-2.5, 3.1-3.9 (Teach Yourself C)

IF STATEMENT



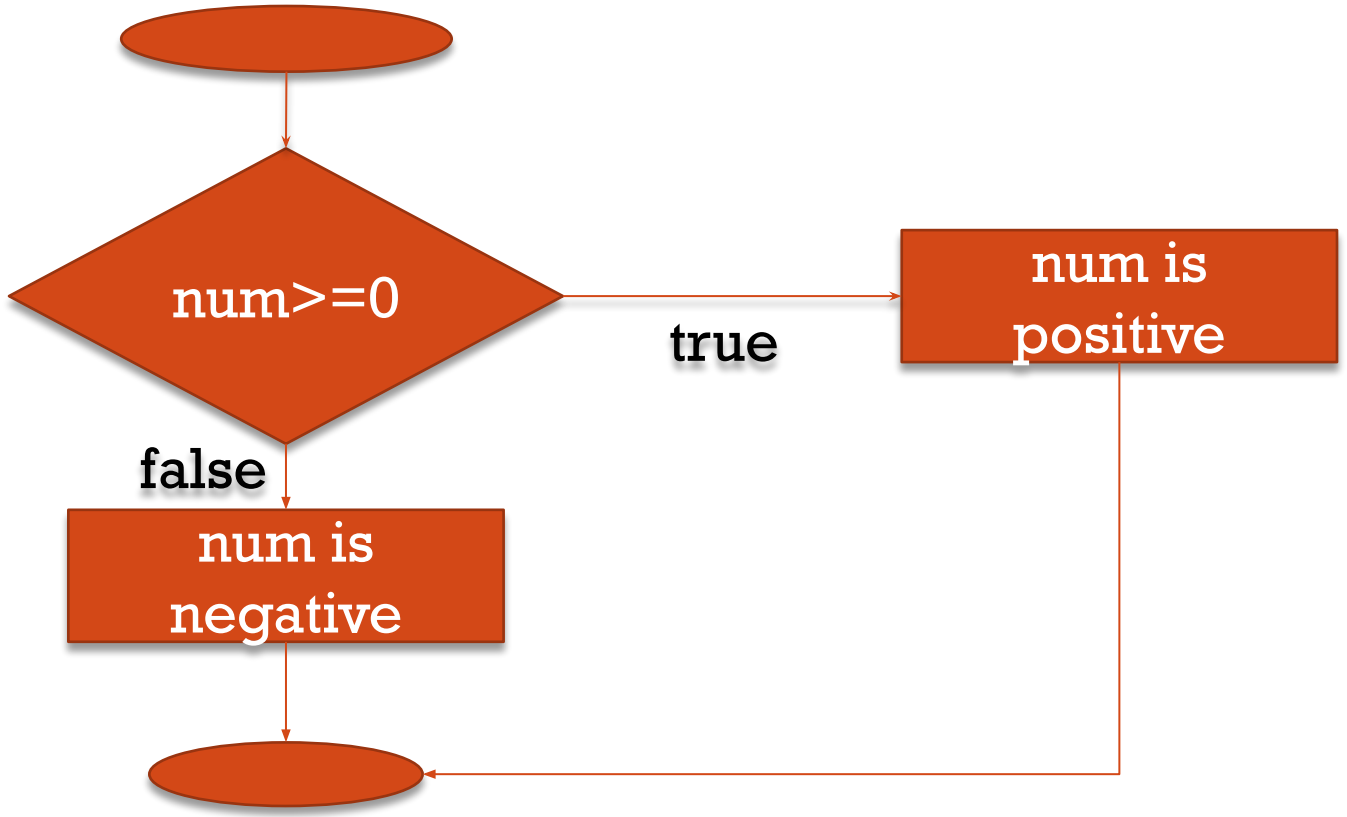
IF STATEMENT

- Selection statement/conditional statement
- Operation governed by outcome of a conditional test
- **if(*expression*) *statement*;**
  - The *expression* can be any valid C expression
  - If *expression* is **true** *statement* will be executed
  - If *expression* is **false** *statement* will be bypassed
  - **true**: any nonzero value
  - **false**: zero
  - Normally *expression* consists of **relational & logical operator**

Let's look at an example …

IF-ELSE STATEMENT

- if(*expression*) *statement1*;
  else *statement2*;
  - If *expression* is **true** *statement1* will be evaluated and *statement1* will be skipped
  - If *expression* is **false** *statement1* will be bypassed and *statement2* will be executed
  - **Under no circumstances both the statements will execute**
  - Two-way decision path



Check if a variable is equal to -1

```c
#include <stdio.h>
int main()
{
    int num=-1;
    if(num+1)
    {
        printf("The number is not -1");
    }
    else
    {
        printf("The number is -1");
    }
}
```

```c
#include <stdio.h>
int main()
{
    int num=-1;
    if(num!=-1)
    {
        printf("The number is not -1");
    }
    else
    {
        printf("The number is -1");
    }
}
```

BLOCKS OF CODE

- Syntax:

```
if(expression) {
    statement1;
    statement2;
     …
    statementN;
 }
else {
    statement1;
    statement2;
   …
    statementN;
 }
```

- If *expression* is **true** all the statements with if will be executed
- If *expression* is **false** all the statements with else will be executed

BLOCKS OF CODE

- Surround the statements in a block with opening and ending with curly braces.
- One indivisible logical unit
- Can be used anywhere a single statement may be used
- Denotes multiple statements
- Common programming error:
  - Forgetting braces of compound statements/blocks

CHECK IF A VARIABLE IS POSITIVE OR NOT

```c
#include <stdio.h>
int main()
{
    int num=-1;
    if(num>=0)
        printf("The number is positive");
    else
        printf("The number is negative");
}
```

No, curly braces needed since if and else contains only one statement.

```c
#include <stdio.h>
int main()
{
    int num=-1;
    if(num>=0)
        printf("Yes");
        printf("The number is positive");
    else
        printf("No");
        printf("The number is negative");
}
```

This one will generate an error since there are multiple statements in the if-else

CHECK IF A VARIABLE IS POSITIVE OR NOT

```c
#include <stdio.h>
int main()
{
    int num=-1;
    if(num>=0)
    {
        printf("Yes");
        printf("The number is positive");
    }
    else
    {
        printf("No");
        printf("The number is negative");
    }
}
```

No, second brackets since if and else contains only one statement. best practice is to always use curly braces

EXAMPLE

Take number of arguments as input. If it is 2 then take 2 number as inputs and sum them, else take 3 numbers as input and sum them.

```
#include<stdio.h>
int main( )
{
 int numOfArg, sum;
 scanf("%d", &numOfArg);
 if(numOfArg==2)
 {
   int a, b;
   scanf("%d %d", &a, &b);
   sum=a+b;
 }
 else
 {
   int a, b, c;
   scanf("%d %d %d", &a, &b, &c);
   sum=a+b+c;
 }
 printf("The sum is %d\n", sum);
 return 0;
}
```

IF STATEMENT

- Common programming errors:
  - Placing ; (semicolon) immediately after condition in **if**
    - if(*expression*); *statement*;
  - Confusing equality operator (==) with assignment operator (=)
    - if(a=b)
    - if(a=5)
    - if(9=5)
    - if(9+5)

```c
#include<stdio.h>
int main()
{
    int a=2;
    if(a=5)
        printf("a=5");
    else
        printf("a in not 5");
}
```

This will print out a=5

IF-ELSE STATEMENT

- **else** part is optional
- The **else** is associated with closest **else**-less **if**
  - ```
    if(n>0)
        if(a>b) z=a;
        else z=b;
    ```
- Where **else** will be associated is shown by indentation
- Braces must be used to force association with the first **if**
  - ```
    if(n>0)
    {
        if(a>b) z=a;
    }
    else z=b;
    ```

NESTED IF (SECTION 3.4)

Take 2 numbers as inputs. Print whether the first one is greater than, less than or equal to the 2nd one

```
#include <stdio.h>
int main() {
    int number1, number2;
    printf("Enter two integers: ");
    scanf("%d %d", &number1, &number2);

    if (number1 >= number2) {
      if (number1 == number2)
        printf("Result: %d = %d",number1,number2);
      else
        printf("Result: %d > %d", number1, number2);
    }
    else
        printf("Result: %d < %d",number1, number2);

    return 0;
}
```

IF-ELSE IF STATEMENT

- Syntax:

```
if(expression)
    statement;
else if (expression)
    statement;
else if (expression)
    statement;
else
    statement;
```

- Multi-way decision
- *expressions* are evaluated **in order**
- If any *expression is true*
  - the *statement* associated with it is executed
  - the whole chain is terminated
- The last **else statement**
  - **Only executed when none of the *expressions* are true**
  - Handles none of the above/ default case
  - Optional
- Multiple *statements* can be associated using curly braces

IF-ELSE IF STATEMENT

Take 2 numbers as inputs. Print whether the first one is greater than, less than or equal to the 2nd one

```
56  #include<stdio.h>
57  int main()
58  {
59      int a,b;
60      scanf("%d %d",&a,&b);
61      if(a==b)
62      printf("the numbers are equal");
63      else if(a<b)
64      printf("first one is smaller");
65      else
66      printf("second one is smaller");
67  }
```

EXAMPLE

```
#include<stdio.h>
int main( )
{
 int num;
 scanf("%d", &num);
 if(num>=75)
   printf(“Grade:A");
 else if(num>=35)
   printf("Grade:B");
 else
   printf("Grade:I");
 return 0;
}
```

CONDITIONAL EXPRESSIONS

- Uses ternary operator " ? : "
- Syntax:

  expression1 ? *expression2* : *expression3*;

- z= (a>b)? a: b; /* z=max(a,b);*/
  - Checks the expression (a>b)
  - If true, assigns a to z
  - If false, assigns b to z
- Can be used anywhere an expression can be

Find the minimum between 3 numbers

```
#include <stdio.h>


int main()
{
   int a,b,c;
   scanf("%d %d %d",&a,&b,&c);
   int min=(a>b)?((b>c)?c:b):((a>c)?c:a);
   printf("%d",min);
}
```

TRY IT YOURSELF…

- Find maximum of three numbers
- Find second maximum of three numbers
- Find minimum of four numbers

SWITCH CASE

- Syntax:

```
switch (expression) {
        case constant: statements
        case constant: statements
        default: statements
}
```

- Use of break

SWITCH CASE – EXAMPLE 1

```
switch (day) {
    case 1: printf(“Sunday\n");
    case 2: printf(“Monday\n");
…
…
    default: printf("Invalid\n");
}
```

SWITCH CASE – EXAMPLE 2

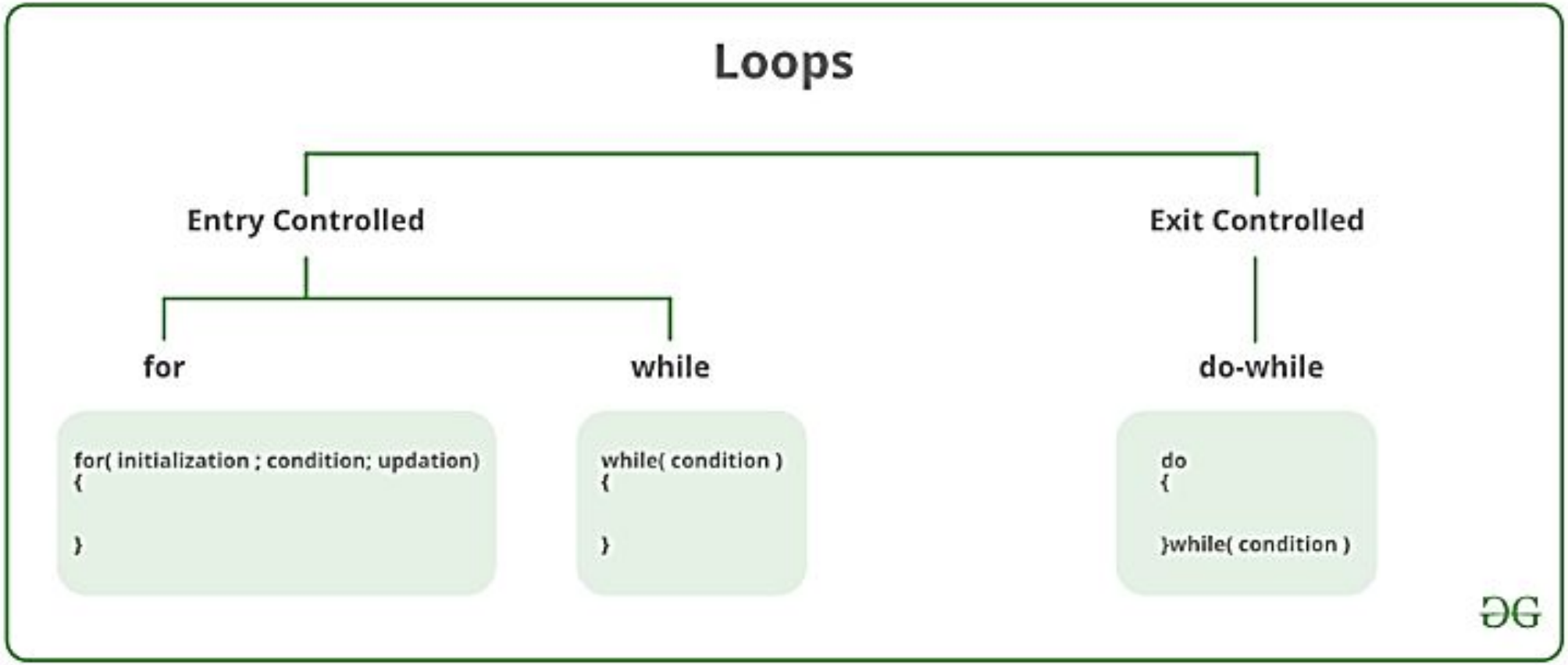
```
#include<stdio.h>
#include<conio.h>
int main()
{
 char i;//getche();
 switch (i) {
   case '0':
   case '1':
   case '2':
   case '3':
   case '4':
   case '5':
   case '6':
   case '7':
   case '8':
   case '9':
      printf(" : digit\n");
      break;
default:
      printf(" : non digit\n");}
return 0;
}
```

SWITCH CASE – EXAMPLE 3

```
//int x=a/b;
int x, a, b;
scanf("%d %d", &a, &b);
switch (b) {
   case 0:
      printf("divide by zero error\n");
      break;
   default: x=a/b;
}
```

LOOP STATEMENTS

- The ability to repeat a block of code a number of times.
- A loop is a way to execute a piece of code repeatedly until the condition is met .

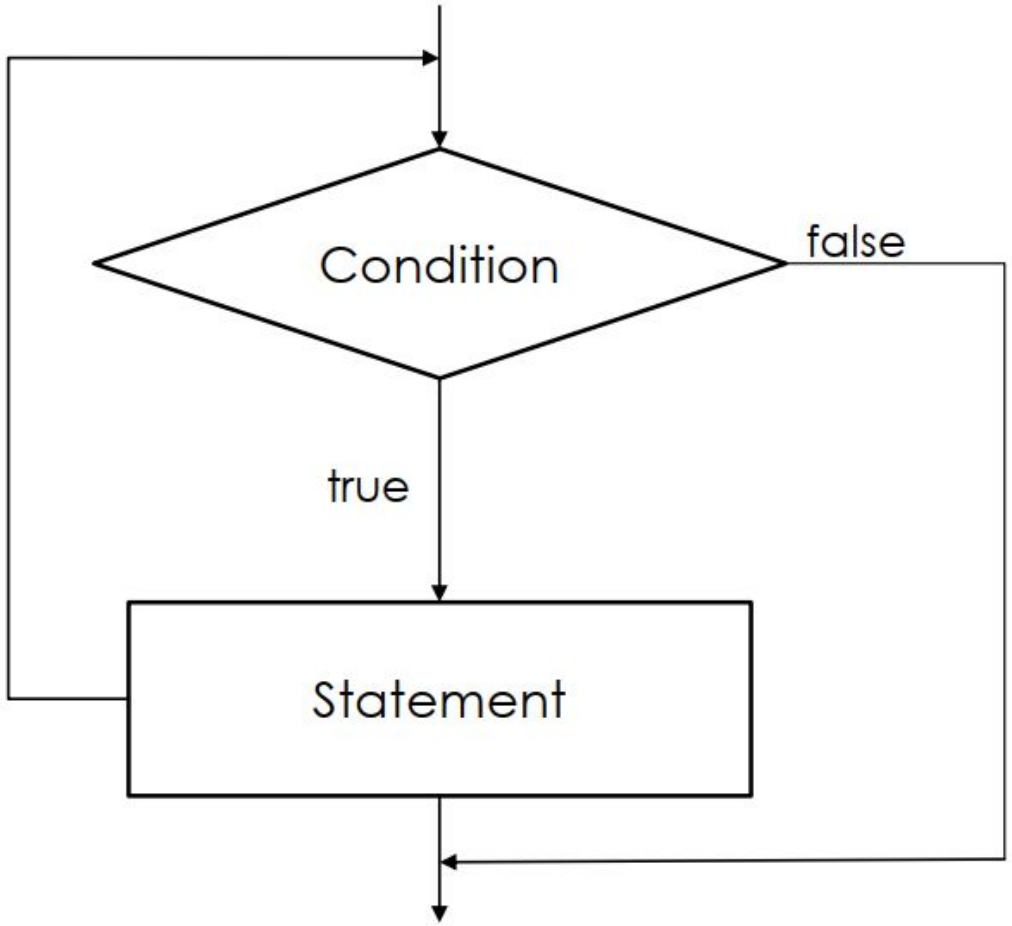


WHILE LOOP

- Syntax:

```
while(conditional_test)
{
    statement(s);
    increment;
}
```

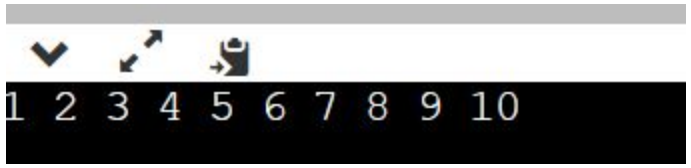
- Will keep on executing the *statements* until the *conditional_test* remains true.



WHILE LOOP - EXAMPLE

```c
#include<stdio.h>
int main()
{
    int i,a;
    i=0;
    while(i<=9)
        {
            i++;
            printf("%d ",i);
        }

}
```




```c
#include<stdio.h>
int main()
{
    int i,a;
    i=10;
    while(i<=9)
        {
            i++;
            printf("%d ",i);
        }

}
```

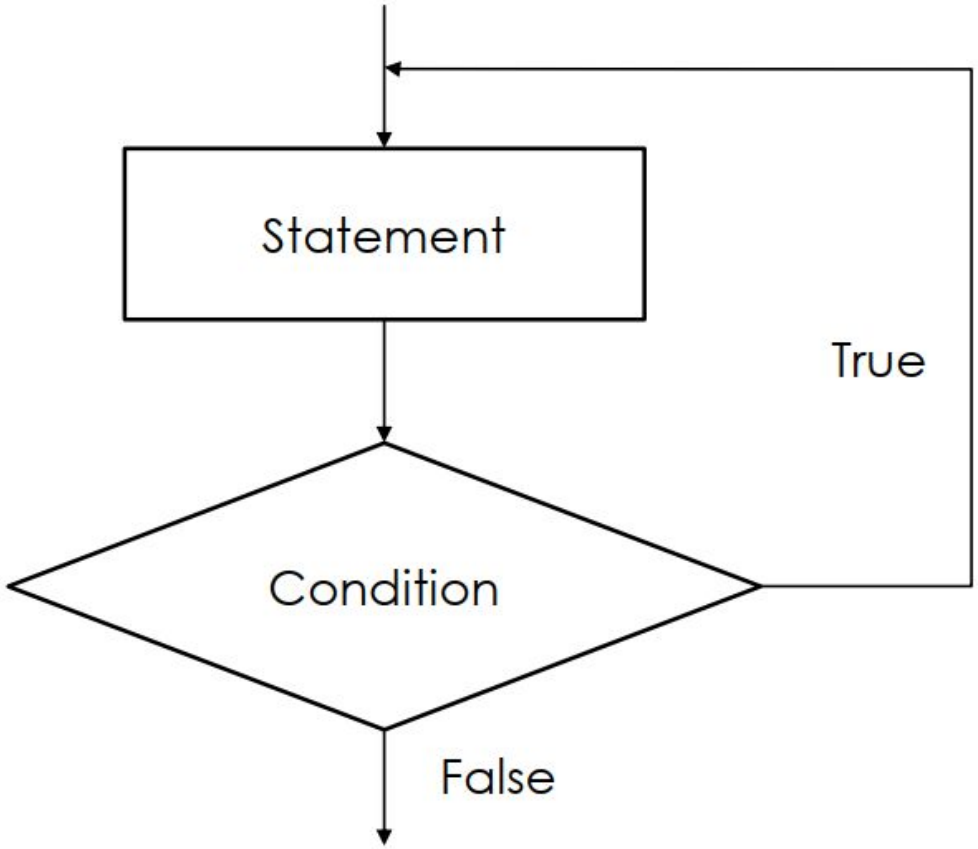
Doesn't enter the loop

DO WHILE LOOP

- Syntax:

```
do
{
    statement;
    increment;
} while(conditional_test);
```

- **Test is at the bottom**
- **Will execute at least once**
- Common error
  - Forgetting the semicolon (;) after while



DO WHILE LOOP - EXAMPLE

```c
#include<stdio.h>
int main()
{
    int i,a;
    i=0;
    do
        {
            i++;
            printf("%d ",i);
        }while(i<=9);

}
```

```
1 2 3 4 5 6 7 8 9 10

...Program finished with exit code 0
Press ENTER to exit console.
```

```c
#include<stdio.h>
int main()
{
    int i,a;
    i=10;
    do
        {
            i++;
            printf("%d ",i);
        }while(i<=9);

}
```

```
11
```

FOR LOOP

- for(*initialization*; *conditional-test*; *increment*) *statement*;
- *initialization*:
  - Give an initial value to the variable that controls the loop (*loop-control variable*)
  - Executed only once
  - Before the loop begins
- *conditional-test:*
  - Tests the *loop-control variable* against a target value
  - If **true** the loop repeats
    - *statement* is executed
  - If **false** the loop stops
    - Next line of code following the loop will be executed
- *increment*:
  - Executed at the bottom of the loop

FOR LOOP – STEP BY STEP

```
for(i=1; i<3; i++)
    printf("%d\n", i);
```

1. i is initialized to 1 (Initialization part is executed only once)
2. Conditional test i<3 is true as i is 1, so the loop executes
3. The value of i will be printed, which is 1
4. Value of i is incremented
5. Conditional test i<3 is true as i is 2, so the loop executes
6. The value of i will be printed, which is 2
7. The value of i will be incremented, so now i is 3.
8. Conditional test i<3 is false as i is 3, so the loop stops

FOR LOOP

Single Statement

```
for(i=1; i<100; i++)
   printf("%d\n", i);
```

- Prints 1 to 99

```
for(i=100; i<100; i++)
   printf("%d\n", i);
```

- This loop will not execute

Block of Statements

```
sum=0;
prod=1;
for(i=1; i<5; i++)
{
  sum+=i;
  prod*=i;
}
printf("sum, prod is %d,
%d\n", sum, prod);
```

FOR LOOP

- **for** loop can run negatively
- ***decrement*** can be used instead of *increment*
  - for(i=20; i>0; i--) …
- Can be incremented or decremented by more than one
  - for(i=1; i<100; i+=5)

FOR LOOP

- All of the following loops will print 1 to 99
  - ```
    for(i=1; i<100; i++)
        printf("%d\n", i);
    ```
  - ```
    for(i=1; i<=99; i++)
        printf("%d\n", i);
    ```
  - ```
    for(i=0; i<99; i++)
        printf("%d\n", i+1);
    ```
  - ```
    for(i=0; i<=98; i++)
        printf("%d\n", i+1);
    ```
- So selection of initial value and loop control condition is important

REVIEW – DIVISIBILITY, FACTORS/DIVISORS, GCD, LCM

GCD of two numbers the largest number by which both the inputs are divisible

FOR LOOP - EXAMPLE

GCD of two numbers (a,b):

```
#include <stdio.h>
int main(void) {
  int a, b, min, i, gcd;
  scanf("%d %d", &a, &b);
  min=(a<b)?a:b;
  for(i=1; i<=min; i++)
     if(a%i==0 && b%i==0)
         gcd=i;
  printf("gcd of %d & %d is %d\n", a, b, gcd);
  return 0;
}
```

if (x%n) evaluates to 0, it means:

- The remainder of dividing x by n will be 0
- n is a divisor/factor of x
- x is a multiple of n

NESTED FOR LOOP

```
#include<stdio.h>

int main()
{
  for(int i=1; i<=3; i++)
  {
    for(int j=1; j<=i; j++)
    {
       printf("%d, %d\n", i, j);
    }
  }

  return 0;
}
```

Output:

1, 1

2, 1

2, 2

3, 1

3, 2

3, 3

What if the condition is j<=3? Try yourself

PRIME NUMBER CHECKER

- Because, the smallest multiple that will not make it a prime is 2. If you have checked all the numbers from 0 to n/2, what multiple is left that could possibly work? If multiple by 2 is bigger than n, then a multiple of 3 or 4 etc will also be bigger than n.
- So the largest factor for any number N must be <= N/2
- So yes take N/2, and check all integers smaller or equal to N/2.

PRIME NUMBER TESTER (WHILE LOOP)

```
/*prime number tester*/
#include<stdio.h>
int main()
{
 int i, num, is_prime=1;
 printf("Enter the number to
 test: ");
 scanf("%d", &num);
 /*test for factors*/
 i=2;
 while(i<=num/2)
 {
   if(!(num%i)) is_prime=0;
   i++;
 }
 if(is_prime)
   printf("%d is prime\n", num);
 else
   printf("%d is not prime\n", num);
 return 0;
}
```

PRIME NUMBER TESTER (DO WHILE LOOP)

```
/*prime number tester*/
#include<stdio.h>
int main()
{
  int i, num, is_prime=1;
  printf("Enter the number to test: ");
  scanf("%d", &num);
  /*test for factors*/
  i=2;
  do
  {
     if(!(num%i)) is_prime=0;
     i++;
   }while(i<=num/2);
  if(is_prime)
     printf("%d is prime\n", num);
  else
     printf("%d is not prime\n", num);
  return 0;
}
```

- It will show that 2 is not prime!!

PRIME NUMBER TESTER (FOR LOOP)

```
#include<stdio.h>
int main()
{
  int i, num, is_prime=1;
  printf("Enter the number to test: ");
  scanf("%d", &num);
  for(i=2; i<=num/2; i++)
     if((num%i)==0) is_prime=0;
  if(is_prime==1)
     printf("%d is prime\n", num);
  else
     printf("%d is not prime\n", num);
  return 0;
}
```

USE OF BREAK

```
/*prime number tester*/
#include<stdio.h>

int main()
{
 int i, num, is_prime=1;
 printf("Enter the number to test: ");
 scanf("%d", &num);
 /*test for factors*/
 for(i=2; i<=num/2; i++)
   if(!(num%i))
   {
       is_prime=0;
       break;
   }
if(is_prime)
   printf("%d is prime\n", num);
else
   printf("%d is not prime\n", num);
return 0;
}
```

CONTINUE IN FOR LOOP

```c
#include<stdio.h>
int main(){
int i=1;//initializing a local variable
//starting a loop from 1 to 10
for(i=1;i<=10;i++){
if(i==5){//if value of i is equal to 5, it will continue the loop
continue;
}
printf("%d \n",i);
}//end of for loop
return 0;
}
```

```
1
2
3
4
6
7
8
9
10
```

USE OF CONTINUE

```
#include<stdio.h>
int main()
{
 for(int i=1; i<=3; i++)
 {
   for(int j=1; j<=i; j++)
   {
      if(i==j) continue;
      printf("%d, %d\n", i, j);
   }
 }
 return 0;
}
```

Output:

2, 1

3, 1

3, 2

CONTINUE WITH WHILE

```c
#include<stdio.h>
void main ()
{
    int i = 0;
    while(i!=10)
    {
        printf("%d", i);
        continue;
        i++;
    }
}
```

- Infinite loop

Find the LCM of 3 numbers

According to Mathematics, the Least Common Multiple of two or more integers is the smallest positive integer that is perfectly divisible by the given integer values without the remainder.

```
#include <stdio.h>

int main() {


    int a,b,c, max,LCM;

    scanf("%d %d %d",&a,&b,&c);



    if(a>b)
    {
        if(a>c)
            max=a;
        else
            max=c;
    }
    else
    {
        if(b>c)
            max=b;
        else
            max=c;
    }
    while (1) {
        if ((max%a==0) && (max%b==0) && (max%c==0)) {
            LCM=max;
            break;
        }
        max++;
    }
    printf("%d",max);
}
```

INPUT CHARACTERS

- getche()/getch()/getchar can be used
- getchar()
  - Compiler dependent
  - waits for carriage return key (**hitting enter**)
  - Read only one char
  - Other input and carriage return will be in buffer
  - Subsequent input (e.g, scanf) will consume them.
  - Defined in stdio.h
- getche()/getch()
  - Return immediately after a key is pressed
  - Defined in conio.h
  - **Its function is to hold the screen until the user passes a single value to exit from the console screen.**

PRACTICE ON LOOP

- Given a number as input write to program to calculate the number of digits.
- Given a number as input write to program to calculate the sum of it's digits.
- Write a program to find x^m where x and m are inputs
- Write a program to convert a decimal number to a binary number (hint: use bitwise operation)

PRACTICE (CONTD.)

- Given the length and breadth of a rectangle, write a program to find whether the area of the rectangle is greater than its perimeter. For example, the area of the rectangle with length = 5 and breadth = 4 is greater than its perimeter.

- Given the coordinates (x, y) of a center of a circle and it's radius, write a program which will determine whether a point (input through the keyboard) lies **inside the circle, on the circle or outside the circle.** [Hint: Use sqrt( ) and pow( ) functions from <math.h>]

- Write a program to determine whether the character entered through the keyboard is **a lower case alphabet** or **an upper case alphabet** or **neither.**

- Write a program using **do-while loop** to test whether a number is prime or not.

- Write a program using **while loop** to find $\mathbf{x^m}$ where x and m are inputs. [You can't use pow() function]